تعريفQuerySet

في Django **QuerySet** هو **مجموعة من الكائنات (objects) التي تم جلبها من قاعدة البيانات** .يمكن اعتباره بمثابة **استعلام (query)مرن وقابل للتعديل قبل التنفيذ الفعلي على قاعدة البيانات**.
كل Model في Django يمكنه إنشاء QuerySet لاسترجاع، تصفية، تعديل، أو حذف البيانات.

الخصائص الأساسية للQuerySet

1. **Lazy Evaluation التقييم الكسول**
   - QuerySetلا يتم تنفيذه مباشرة عند تعريفه.
   - يتم التنفيذ عند الحاجة فعليًا، مثل عند الوصول للبيانات أو تحويله لقائمة.
   - مثال:
   - qs = Student.objects.all() # لم يتم تنفيذ الاستعلام بعد
   - list(qs) # هنا يتم تنفيذ الاستعلام فعليًا
2. **قابلية التصفية والتسلسل**
   - يمكنك تطبيق عدة شروط على:QuerySet
     - filter()
     - exclude()
     - order_by()
     - distinct()
   - كل استدعاء لهذه الدوال يرجع **QuerySetجديد**.
3. **التكرار(Iterable)**
   - يمكنك تكرار QuerySet مثل قائمة:
   - for student in Student.objects.all():
   - print(student.f_name)
4. **الاتصال بسلاسل الاستعلام(Chaining)**
   - يمكن ربط الاستدعاءات:
   - Student.objects.filter(age__gte=18).exclude(level='4').order_by('f_name')

طرق إنشاءQuerySet

3.1جلب كل السجلات

```
students = Student.objects.all()
```

3.2التصفية

```
adults = Student.objects.filter(age__gte=18)
```

3.3استبعاد سجلات معينة

```
non_fourth_level = Student.objects.exclude(level='4')
```

3.4ترتيب النتائج

```
# students_sorted = Student.objects.order_by('f_name') ترتيب تصاعدي
# students_sorted_desc = Student.objects.order_by('-f_name') ترتيب تنازلي
```

3.5الحصول على سجل واحد

```
student = Student.objects.get(pk=1)
```

ملاحظة get() :يرجع سجل واحد فقط وإذا لم يوجد أو كان أكثر من سجل يرفع خطأ.

3.6العد

```
count = Student.objects.filter(level='1').count()
```

3.7استخدام التجميعAggregation

```
from django.db.models import Avg  Max  Min

avg_age = Student.objects.all().aggregate(Avg('age'))
max_age = Student.objects.all().aggregate(Max('age'))
```

3.8استخدام القيمValue & Values_list

```
names = Student.objects.values('f_name'  'l_name')  # dicts
# names_list = Student.objects.values_list('f_name'  flat=True) قائمة
```

طرق تحسين أداء QuerySet

1. **select_related**: لتحسين استعلامات العلاقات OneToOne أوForeignKey

```
students = Student.objects.select_related('school').all()
```

2. **prefetch_related**: لتحسين استعلامات ManyToMany أو علاقات متعددة

```
students = Student.objects.prefetch_related('courses').all()
```

3. **only / defer**: لجلب أعمدة محددة فقط

```
students = Student.objects.only('f_name'  'l_name')
# students = Student.objects.defer('address') تجاهل عمود
```

المميزات المهمة للـ QuerySet

- **قابلية التعديل قبل التنفيذ** :يمكن إضافة أو تعديل شروط بسهولة.
- **إعادة الاستخدام** :يمكن تخزين QuerySet وإعادة استخدامه.
- :**Lazy Evaluation**تحسين الأداء بتأجيل تنفيذ الاستعلام.
- **قابلية السلسلة** :يمكن ربط عدة دوال للحصول على استعلام متقدم.
- **تكامل مع** :**ORM** التعامل مع البيانات بطريقة كائنية دون الحاجة لكتابة SQL صريح.

مثال شامل

لنفترض وجود نموذج:Student

```
class Student(models.Model):
f_name = models.CharField(max_length=20)
l_name = models.CharField(max_length=20)
age = models.IntegerField()
level = models.CharField(max_length=2)
```

مثال استعلام شامل:

```
from django.db.models import Avg

#جلب الطلاب البالغين في الصف 3 وترتيبهم تصاعديًا
qs = Student.objects.filter(age__gte=18  level='3').order_by('f_name')

#حساب متوسط أعمارهم
avg_age = qs.aggregate(Avg('age'))

#طباعة النتائج
for student in qs:
print(student.f_name  student.age)

print("Average Age:"  avg_age['age__avg'])
```